Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №12

Дисциплина: Основы программирования на Python

Тема: Работа с реляционными базами данных в Python с использованием SQLAlchemy и ORM

Вариант 2: Библиотека

Цель работы

Получить практические навыки разработки Python-приложений, взаимодействующих с реляционной базой данных посредством ORM-библиотеки SQLAlchemy. В ходе работы необходимо изучить основные принципы ORM, освоить SQLAlchemy, научиться описывать модели базы данных в виде Python-классов, выполнять операции создания, чтения, изменения и удаления данных, устанавливать связи между сущностями, выполнять фильтрацию, сортировку и поиск, использовать транзакции и организовать разделение моделей, доступа к данным и бизнес-логики.

Постановка индивидуального задания (Вариант 2. Библиотека)

Разработать приложение библиотеки с сущностями Reader (читатель), Book (книга) и Loan (выдача книги). Реализовать выдачу и возврат книги читателем, а также полный набор операций CRUD для книг и читателей, поиск, фильтрацию, сортировку и статистику фонда. Приложение должно взаимодействовать с реляционной базой данных через ORM SQLAlchemy, содержать разделение на модели, репозитории и сервисный слой, использовать транзакции и обрабатывать ошибки предметной области.

Краткие теоретические сведения

ORM (Object-Relational Mapping) — технология отображения объектов объектно-ориентированной программы на таблицы реляционной базы данных. Без ORM разработчик формирует SQL-запросы вручную и самостоятельно преобразует строки результата в объекты, что приводит к дублированию кода и ошибкам при изменении схемы. ORM берёт это преобразование на себя: класс соответствует таблице, атрибут класса — столбцу, экземпляр класса — строке.

В SQLAlchemy версии 2.0 модели описываются через класс DeclarativeBase и типизированные аннотации Mapped, а столбцы объявляются функцией mapped_column(). Связи задаются через relationship() с параметром back_populates, что обеспечивает двусторонний доступ: от выдачи к книге и от книги к списку выдач. Ограничения целостности объявляются в __table_args__ с помощью CheckConstraint и UniqueConstraint, а внешние ключи — через ForeignKey с указанием поведения при удалении.

Работа с данными выполняется через объект Session, который отслеживает изменения объектов и накапливает их до фиксации транзакции. Метод flush() отправляет накопленные изменения в базу в рамках текущей транзакции, а commit() фиксирует их окончательно; при возникновении исключения транзакция откатывается. Запросы строятся функциями select(), where(), order_by(), offset() и limit(), а агрегаты вычисляются функциями func.count(), func.sum() и func.avg().

ORM позволяет работать с любой совместимой СУБД: строка подключения задаётся в формате диалект+драйвер://пользователь:пароль@хост/база. Приложение использует SQLite для локального запуска и может быть переключено на PostgreSQL изменением переменной окружения DATABASE_URL без правки кода.

Проектирование базы данных

База данных состоит из трёх таблиц. Связь между читателем и книгой имеет тип «многие ко многим» и реализована через таблицу выдач loans, которая одновременно хранит даты выдачи, срока возврата и фактического возврата.

Таблица	Поля	Ключи и ограничения
readers	id, full_name, email, phone, registered_at	PK id, UNIQUE email, дата регистрации по умолчанию
books	id, title, author, genre, year, price, copies_total, copies_available	PK id, CHECK на год, цену и соотношение экземпляров
loans	id, book_id, reader_id, loan_date, due_date, return_date	PK id, FK на books и readers, UNIQUE по книге, читателю и дате, CHECK на порядок дат

Связи описаны отношениями Reader.loans и Book.loans с каскадным удалением: при удалении читателя или книги связанные выдачи удаляются автоматически. Ограничение copies_available <= copies_total на уровне базы данных дополняет проверку в сервисном слое и защищает от рассогласования данных даже при прямом обращении к базе.

Структура проекта

Компонент	Назначение
app/database.py	Базовый класс Base, создание движка, включение внешних ключей для SQLite, фабрика сессий
app/models/entities.py	ORM-модели Reader, Book, Loan с аннотациями типов и связями
app/repositories/repositories.py	Репозитории BookRepository, ReaderRepository, LoanRepository
app/services/services.py	Класс LibraryService: бизнес-логика, валидация, транзакции
app/errors.py	Иерархия ошибок: AppError, NotFoundError, ValidationError, DuplicateError, ForeignKeyError, TransactionError
app/seed.py	Заполнение базы демонстрационными данными
app/main.py	Демонстрационный и интерактивный режимы консольного приложения
tests/	Тесты репозиториев и сервисного слоя

ORM-модели

Модели описаны в декларативном стиле SQLAlchemy 2.0. Класс Book содержит строковые поля названия, автора и жанра, год издания, цену в типе

Numeric с двумя знаками после запятой и два счётчика экземпляров. В __table_args__ объявлены три ограничения CHECK, которые дублируют правила предметной области на уровне базы данных. Класс Loan связывает книгу и читателя внешними ключами и хранит даты выдачи, срока возврата и возврата; для него задано уникальное ограничение по трём полям, исключающее повторную запись о выдаче одной книги тому же читателю в тот же день.

Листинг 12.1 — ORM-модель книги (app/models/entities.py)

```
class Book(Base):
    __tablename__ = "books"
    __table_args__ = (
        CheckConstraint("year >= 1400 AND year <= 2100", name="ck_books_year"),
        CheckConstraint("price >= 0", name="ck_books_price"),
        CheckConstraint("copies_available >= 0 AND copies_available <= copies_total",
                        name="ck_books_copies"),
    )

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200), nullable=False)
    author: Mapped[str] = mapped_column(String(120), nullable=False)
    genre: Mapped[str] = mapped_column(String(60), nullable=False)
    year: Mapped[int] = mapped_column(nullable=False)
    price: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
    copies_total: Mapped[int] = mapped_column(nullable=False)
    copies_available: Mapped[int] = mapped_column(nullable=False)

    loans: Mapped[List["Loan"]] = relationship(
        back_populates="book", cascade="all, delete")
```

Слой доступа к данным

Каждый репозиторий получает сессию в конструкторе и инкапсулирует запросы к своей сущности. BookRepository реализует добавление, получение по идентификатору, список с фильтрацией по цене и наличию, сортировкой по выбранному полю и пагинацией через offset и limit, поиск по названию, автору и жанру с помощью оператора ilike, поиск по автору и статистику фонда. ReaderRepository дополнительно умеет находить читателя по электронной почте. LoanRepository создаёт запись о выдаче, возвращает выдачи читателя, активные и просроченные выдачи, а также проверяет наличие активной выдачи конкретной книги у конкретного читателя.

Для сортировки используется словарь SORT_FIELDS, сопоставляющий строковое имя поля столбцу модели; это позволяет принимать имя сортировки

из пользовательского ввода, не подставляя его напрямую в SQL. Связанные объекты книги и читателя загружаются заранее функцией selectinload(), что избавляет от проблемы N+1 запроса при выводе списка выдач.

Листинг 12.2 — Фильтрация, сортировка и пагинация (repositories.py)

```
SORT_FIELDS = {
    "title": Book.title,
    "author": Book.author,
    "year": Book.year,
    "price": Book.price,
}

def list(self, sort: str = "title", descending: bool = False,
         offset: int = 0, limit: int = 50,
         min_price: Optional[Decimal] = None,
         available_only: bool = False) -> Sequence[Book]:
    stmt = select(Book)
    if min_price is not None:
        stmt = stmt.where(Book.price >= min_price)
    if available_only:
        stmt = stmt.where(Book.copies_available > 0)

    column = SORT_FIELDS.get(sort, Book.title)
    stmt = stmt.order_by(column.desc() if descending else column.asc(),
                         Book.id)
    stmt = stmt.offset(offset).limit(limit)
    return self.session.scalars(stmt).all()
```

Сервисный слой и бизнес-логика

Класс LibraryService объединяет репозитории и отвечает за правила предметной области. Метод add_book() проверяет обязательные поля, корректность года издания, неотрицательность цены и соотношение общего и доступного числа экземпляров, после чего создаёт запись. Метод update_book() принимает произвольный набор полей и изменяет только переданные атрибуты. Методы add_reader() и update-операции проверяют формат электронной почты и её уникальность, преобразуя нарушение ограничения базы данных в понятное исключение DuplicateError.

Выдача книги выполняется методом issue_book(): проверяется существование книги и читателя, наличие доступных экземпляров, отсутствие активной выдачи этой книги данному читателю и корректность срока. Затем уменьшается счётчик доступных экземпляров и создаётся запись о выдаче. Возврат выполняется методом return_loan(): проверяется, что выдача существует

и ещё не закрыта, после чего проставляется дата возврата и увеличивается счётчик доступных экземпляров. Обе операции выполняются в одной транзакции, поэтому при любой ошибке изменения откатываются полностью.

Листинг 12.3 — Выдача книги в транзакции (app/services/services.py)

```
def issue_book(self, book_id: int, reader_id: int, days: int = 14) -> Loan:
    if not 1 <= days <= 90:
        raise ValidationError("Срок выдачи должен быть от 1 до 90 дней")

    with self.database.session() as session:
        try:
            books = BookRepository(session)
            readers = ReaderRepository(session)
            loans = LoanRepository(session)

            book = books.get(book_id)
            if book is None:
                raise NotFoundError("Book", book_id)
            reader = readers.get(reader_id)
            if reader is None:
                raise NotFoundError("Reader", reader_id)
            if book.copies_available <= 0:
                raise ValidationError("Нет доступных экземпляров")
            if loans.active_for(book, reader) is not None:
                raise ValidationError("Книга уже выдана этому читателю")

            book.copies_available -= 1
            loan = loans.add(book, reader, days=days)
            session.commit()
            return loan
        except AppError:
            session.rollback()
            raise
        except Exception as exc:
            session.rollback()
            raise TransactionError(str(exc)) from exc
```

Обработка ошибок

Ошибки предметной области описаны отдельной иерархией. Базовый класс AppError объединяет все ожидаемые ситуации, NotFoundError сообщает об отсутствии сущности с указанием её типа и идентификатора, ValidationError — о нарушении правил проверки, DuplicateError — о конфликте уникальности, ForeignKeyError — о невозможности удалить связанную запись, TransactionError — о сбое транзакции. Консольный интерфейс перехватывает AppError и выводит понятное сообщение, не прерывая работу программы, а непредвиденные исключения преобразуются в TransactionError с сохранением исходной причины через конструкцию raise ... from.

Раздел тестирования

Тесты выполняются на временной базе данных SQLite, создаваемой для каждого теста, что обеспечивает полную изоляцию. Тесты репозиториев проверяют корректность запросов, тесты сервиса — бизнес-правила и обработку ошибок.

Файл тестов	Кол-во	Что проверяется
tests/test_repository.py	10	Добавление и чтение книги, связи с выдачами, фильтрация и сортировка, поиск, удаление, статистика, уникальность почты читателя, активные и просроченные выдачи
tests/test_service.py	20	Создание книги и валидация полей, получение и отсутствие книги, поиск и пустой запрос, изменение и удаление, регистрация читателя и дубликат почты, выдача и возврат книги, отсутствие экземпляров, некорректный срок, повторный возврат, просроченные выдачи, статистика

Все тридцать тестов проходят успешно. Дополнительно приложение проверялось вручную в демонстрационном режиме: база заполнялась тестовыми данными, выводился каталог с сортировкой по цене и фильтром по наличию, выполнялся поиск по автору, оформлялась выдача книги читателю, затем возврат, после чего выводились активные и просроченные выдачи и статистика фонда.

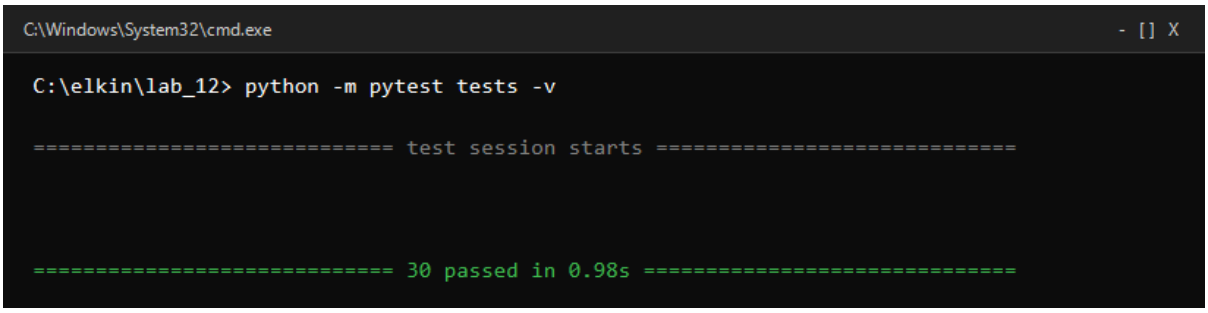


Рисунок 12.1 — Результат прохождения тестов репозиториев и сервисного слоя

Листинги ключевых фрагментов программы

Листинг 12.4 — Подключение к базе данных (app/database.py)

```
DEFAULT_DATABASE_URL = "sqlite:///library.db"

class Base(DeclarativeBase):
    pass
```

```
class Database:
    def __init__(self, url: Optional[str] = None) -> None:
        self.url = url or os.getenv("DATABASE_URL", DEFAULT_DATABASE_URL)
        self.engine = self._create_engine(self.url)
        self.session_factory = sessionmaker(bind=self.engine,
                                            expire_on_commit=False)

    def create_all(self) -> None:
        Base.metadata.create_all(self.engine)

    def session(self) -> Session:
        return self.session_factory()
```

Листинг 12.5 — Поиск и статистика фонда (repositories.py)

```
def search(self, query: str) -> Sequence[Book]:
    pattern = f"%{query}%"
    stmt = (
        select(Book)
        .where(or_(Book.title.ilike(pattern),
                   Book.author.ilike(pattern),
                   Book.genre.ilike(pattern)))
        .order_by(Book.title)
    )
    return self.session.scalars(stmt).all()


def statistics(self) -> dict:
    stmt = select(
        func.count(Book.id),
        func.coalesce(func.sum(Book.copies_total), 0),
        func.coalesce(func.sum(Book.copies_available), 0),
        func.coalesce(func.avg(Book.price), 0),
        func.coalesce(func.min(Book.price), 0),
        func.coalesce(func.max(Book.price), 0),
    )
    count, total, available, avg_price, min_p, max_p = \
        self.session.execute(stmt).one()
    return {"books": int(count), "copies_total": int(total),
            "copies_available": int(available), "avg_price": avg_price,
            "min_price": min_p, "max_price": max_p}
```

Вывод

В ходе выполнения лабораторной работы разработано приложение библиотеки, взаимодействующее с реляционной базой данных через ORM SQLAlchemy. Модели описаны декларативно с аннотациями типов, связями и ограничениями целостности на уровне базы. Разделение на репозитории и сервисный слой позволило изолировать построение запросов от правил предметной области, а использование сессий и транзакций обеспечило согласованность данных при выдаче и возврате книг. Реализованы операции CRUD, поиск, фильтрация, сортировка, пагинация и статистика, а тридцать автоматических тестов подтверждают корректность работы приложения.